



ICT 2308 DATABASE SYSTEMS (Practical)

Ms. Dinushika Wickramasiri
Email: dinushiw@tec.rjt.ac.lk

**Department of Information and Communication Technology
Faculty of Technology
Rajarata University of Sri Lanka**

Content

1. SQL Joins
2. SQL Alias

Lesson 05

SQL Joins

SQL Joins

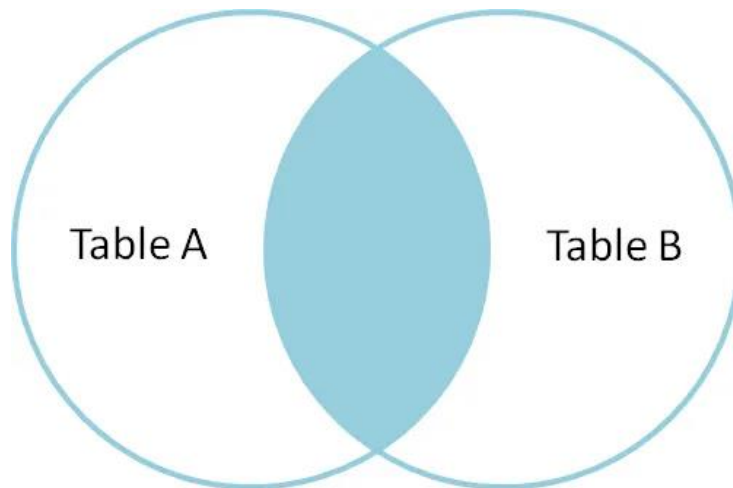
- A JOIN clause is used to combine rows from two or more tables, based on a related column between them.
- There are many types of Joins in SQL. Depending on the use case, you can use different type of SQL JOIN clause.
- Here are the frequently used SQL JOIN types:
 - INNER JOIN
 - LEFT JOIN
 - RIGHT JOIN
 - FULL JOIN
 - NATURAL JOIN

INNER Join

- The INNER JOIN keyword selects all rows from both the tables as long as the condition is satisfied.
- This keyword will create the result-set by combining all rows from both the tables where the condition satisfies i.e value of the common field will be the same.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,... FROM table1 INNER JOIN table2 ON table1.matching_column = table2.matching_column;
```



Note:

We can also write **JOIN** instead of **INNER JOIN**. **JOIN** is same as **INNER JOIN**.

Example - INNER Join

- First, create a database named “School”.
- Consider the two tables, Student and StudentCourse, which share a common column ROLL_NO. Using SQL JOINS, we can combine data from these tables based on their relationship, allowing us to retrieve meaningful information like student details along with their enrolled courses.

Student

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

StudentCourse

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

Example - INNER Join

- Show the names and age of students enrolled in different courses.

Query

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE FROM Student  
INNER JOIN StudentCourse ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

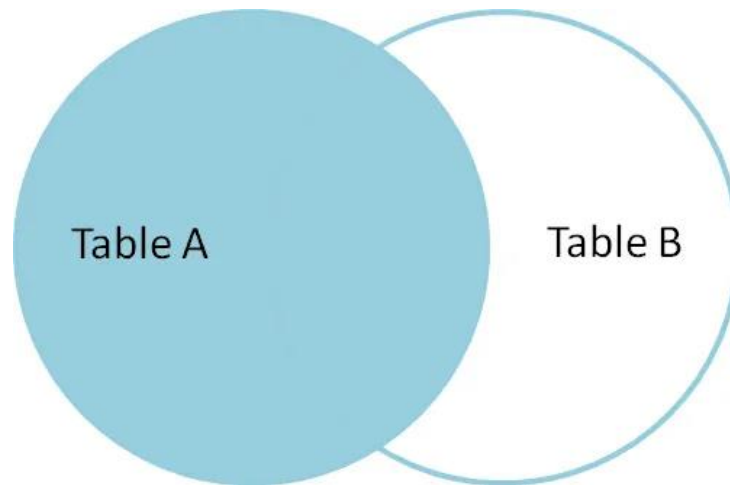
COURSE_ID	NAME	Age
1	HARSH	18
2	PRATIK	19
2	RIYANKA	20
3	DEEP	18
1	SAPTARHI	19

SQL LEFT JOIN

- A LEFT JOIN returns all rows from the left table, along with matching rows from the right table.
- If there is no match, NULL values are returned for columns from the right table.
- **LEFT JOIN** is also known as **LEFT OUTER JOIN**.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,... FROM table1 LEFT  
JOIN table2 ON table1.matching_column = table2.matching_column;
```



Note:

We can also use **LEFT OUTER JOIN** instead of **LEFT JOIN**, both are the same.

Example – LEFT Join

- Show the names of all students along with the course IDs they are enrolled in.

Query

```
SELECT Student.NAME, StudentCourse.COURSE_ID  
FROM Student  
LEFT JOIN StudentCourse  
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Note:

In this example, the LEFT JOIN retrieves all rows from the Student table and the matching rows from the StudentCourse table based on the ROLL_NO column.

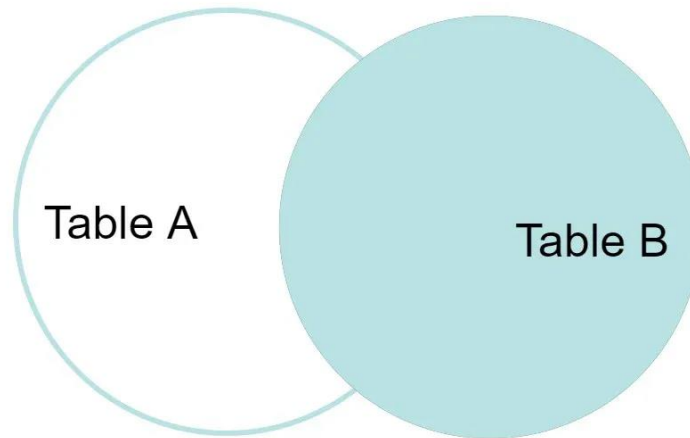
NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
DHANRAJ	NULL
ROHIT	NULL
NIRAJ	NULL

SQL RIGHT JOIN

- RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join.
- It is very similar to LEFT JOIN for the rows for which there is no matching row on the left side, the result-set will contain null.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,... FROM table1 RIGHT JOIN table2 ON table1.matching_column = table2.matching_column;
```



Note:

RIGHT JOIN is also known as **RIGHT OUTER JOIN**.

Example – RIGHT Join

- Show the names of all students along with the course IDs they are enrolled in.

Query

```
SELECT Student.NAME, StudentCourse.COURSE_ID  
FROM Student  
RIGHT JOIN StudentCourse  
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Note:

In this example, the RIGHT JOIN retrieves all rows from the StudentCourse table and the matching rows from the Student table based on the ROLL_NO column.

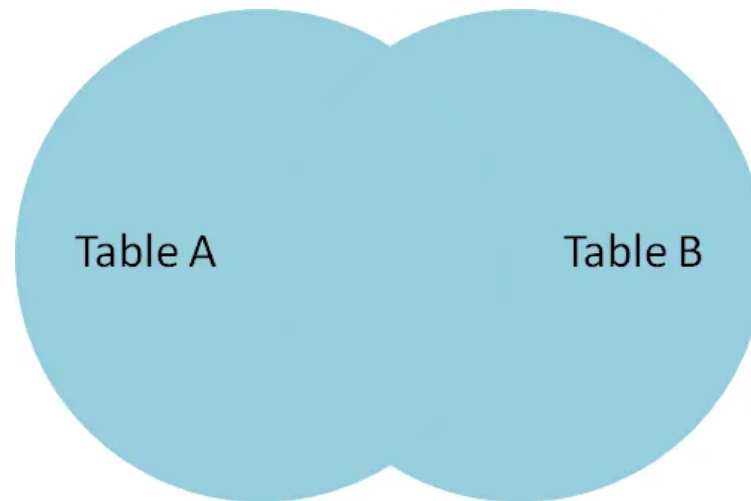
NAME	COURSE_ID
HARSH	1
PRATIK	2
RIYANKA	2
DEEP	3
SAPTARHI	1
NULL	4
NULL	5
NULL	4

SQL FULL JOIN

- FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN.
- The result-set will contain all the rows from both tables.
- For the rows for which there is no matching, the result-set will contain NULL values.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,... FROM table1 FULL JOIN table2 ON table1.matching_column = table2.matching_column;
```



Example – FULL Join

- Show the names of all students along with the course IDs they are enrolled in.

Query

```
SELECT Student.NAME, StudentCourse.COURSE_ID  
FROM Student  
FULL JOIN StudentCourse  
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Note:

This example demonstrates the use of a FULL JOIN, which combines the results of both LEFT JOIN and RIGHT JOIN. The query retrieves all rows from the Student and StudentCourse tables. If a record in one table does not have a matching record in the other table, the result set will include that record with NULL values for the missing fields

* MySQL/MariaDB does not directly support FULL OUTER JOIN. You can simulate it using LEFT JOIN + RIGHT JOIN with UNION

SQL Natural Join

- Natural join can join tables based on the common columns in the tables being joined.
- A natural join returns all rows by matching values in common columns having same name and data type of columns and that column should be present in both tables.
- Both table must have at least one common column with same column name and same data type.
- The two table are joined using **Cross join**.
- DBMS will look for a common column with same name and data type Tuples having exactly same values in common columns are kept in result.

Syntax

```
SELECT * FROM TABLE1 NATURAL JOIN TABLE2;
```

Example – NATURAL Join

- Show the names of all students along with the course IDs.

Query

```
SELECT *  
FROM Student  
NATURAL JOIN StudentCourse;
```

SQL Alias

- In SQL, aliases are temporary names assigned to columns or tables to improve readability and simplify complex queries.
- It does not change the actual table or column name in the database; it's just for that one query.
- There are two types of aliases in SQL:
 1. **Column Aliases:** Temporary names for columns in the result set.
 2. **Table Aliases:** Temporary names for tables used within a query.

Column Aliases

- A column alias is used to rename a column just for the output of a query. They are useful when:
 - ✓ Displaying aggregate data
 - ✓ Making results more readable
 - ✓ Performing calculations

Syntax:

```
SELECT column_name AS alias_name  
FROM table_name;
```

Table Aliases

- A table alias is used when you want to give a table a temporary name for the duration of a query.
- Table aliases are especially helpful in JOIN operations to simplify queries, particularly when the same table is referenced multiple times (like in self-joins).

Query:

```
SELECT c1.CustomerName, c1.Country  
FROM Customer AS c1, Customer AS c2  
WHERE c1.Age = c2.Age AND c1.Country = c2.Country;
```

Example – FULL Join

- Show the names of all students along with the course IDs they are enrolled in.

Query

```
SELECT S.NAME, C.COURSE_ID  
FROM Student S  
LEFT JOIN StudentCourse C ON C.ROLL_NO = S.ROLL_NO  
UNION ALL  
SELECT S.NAME, C.COURSE_ID  
FROM Student S  
RIGHT JOIN StudentCourse C ON C.ROLL_NO = S.ROLL_NO  
WHERE S.ROLL_NO IS NULL;
```

Q&A



THANK YOU